

UNIVERSIDAD INTERNACIONAL DEL ECUADOR

FACULTAD DE INGENIERÍAS DIGITALES Y TECNOLOGÍAS EMERGENTES

ESCUELA DE CIENCIAS DE LA COMPUTACIÓN

EVALUACION CONTACTO CON EL DOCENTE

PROYECO FINAL

NOMBRE

GARCÍA QUESPAS ANABEL KARINA

DOCENTE

AMAN RAMOS LILIAN MARLENE

CARRERA

INGENIERÍA EN SISTEMAS DE INFORMACIÓN

ASIGNATURA

LOGICA DE PROGRAMACION

SEMESTRE

PRIMER SEMESTRE

QUITO – JUNIO 2026

REINVENTEMOS
EL FUTURO

INTRODUCCIÓN

El presente informe tiene como finalidad documentar el desarrollo del proyecto integrador realizado durante la asignatura Lógica de Programación de la carrera de Ingeniería en Sistemas de Información de la Universidad Internacional del Ecuador. El proyecto consistió en el diseño, análisis, implementación y documentación de un juego interactivo del Ahorcado, desarrollado utilizando el lenguaje de programación Python y aplicando los conocimientos adquiridos durante las ocho semanas del semestre.

El desarrollo del proyecto permitió aplicar los fundamentos de la programación estructurada, iniciando con el análisis del problema y la definición de los requerimientos del sistema. Posteriormente se elaboraron los diagramas de casos de uso, la arquitectura del software y el pseudocódigo utilizando PSeInt, herramientas que facilitaron la planificación antes de comenzar la implementación.

Durante las siguientes semanas se desarrolló el programa incorporando variables, estructuras condicionales, ciclos repetitivos, funciones, listas, tuplas y diccionarios. Además, se implementó un sistema de categorías y niveles de dificultad que permite al usuario interactuar con el juego de forma dinámica. Finalmente, el proyecto fue almacenado y administrado mediante GitHub para mantener un adecuado control de versiones.

El resultado obtenido fue un software funcional que permite al usuario seleccionar una categoría, elegir un nivel de dificultad, descubrir una palabra oculta mediante el ingreso de letras y utilizar pistas cuando sea necesario, controlando los intentos disponibles hasta finalizar la partida.

OBJETIVO GENERAL

Desarrollar un juego del ahorcado utilizando Python, aplicando los conocimientos adquiridos durante la asignatura de Lógica de Programación mediante el análisis, diseño, implementación, pruebas y documentación del software.

OBJETIVOS ESPECÍFICOS

- Analizar el problema y definir los requerimientos del software.
- Elaborar el diagrama de casos de uso, arquitectura y diagrama de flujo del sistema.
- Diseñar el algoritmo utilizando pseudocódigo en PSeInt.
- Implementar el programa mediante programación estructurada utilizando Python.
- Aplicar variables, condicionales, ciclos, funciones y estructuras de datos.
- Gestionar el desarrollo mediante GitHub utilizando control de versiones.

- Evaluar el funcionamiento del software mediante diferentes pruebas.

DESARROLLO DEL PROYECTO

Semana 1 – Análisis del problema y planificación

Durante la primera semana se realizó el análisis del problema y la planificación general del proyecto. Se definió que el software a desarrollar sería un juego del ahorcado ejecutado desde la consola, cuyo objetivo principal consiste en que el jugador descubra una palabra oculta antes de agotar el número máximo de intentos permitidos.

En esta etapa se identificaron los actores del sistema, las funcionalidades principales y los requerimientos funcionales del software. Asimismo, se elaboró el diagrama de casos de uso para representar gráficamente la interacción entre el usuario y el sistema. También se diseñó la arquitectura del software, organizando el programa en módulos independientes para facilitar su desarrollo y mantenimiento.

Semana 2 – Diseño del algoritmo

Durante la segunda semana se diseñó el algoritmo mediante pseudocódigo utilizando PSeInt. En esta fase se estableció la lógica principal del juego, incluyendo el menú principal, la selección de categorías, la selección de dificultad, el ingreso de letras, el sistema de pistas, la verificación de victoria o derrota y la posibilidad de reiniciar la partida.

El pseudocódigo permitió validar la lógica del programa antes de iniciar la programación en Python, reduciendo errores y facilitando la comprensión del funcionamiento del software.

También se instaló y configuró el entorno de desarrollo con Python, Visual Studio Code y GitHub, herramientas necesarias para la implementación del proyecto.

Semana 3 – Desarrollo del programa

Durante la tercera semana comenzó la implementación del programa utilizando el lenguaje Python.

Inicialmente se desarrolló el menú principal, permitiendo al usuario iniciar una partida o salir del programa. Posteriormente se implementó el sistema de categorías utilizando diccionarios, donde cada categoría contiene una lista de palabras relacionadas.

Además, se incorporó la selección de dificultad mediante tuplas, permitiendo clasificar las palabras según su longitud. Se desarrollaron funciones independientes para cada proceso del sistema, logrando una estructura modular que facilita futuras modificaciones.

Finalmente, se realizaron las primeras pruebas verificando el correcto funcionamiento del menú, la selección de categorías y la generación aleatoria de palabras.

Semana 4 – Implementación de Variables y Estructuras Condicionales

Durante la cuarta semana se inició la programación de la lógica principal del juego utilizando el lenguaje Python. En esta etapa se implementaron las variables necesarias para almacenar la información utilizada durante cada partida.

Las variables permiten guardar datos que cambian durante la ejecución del programa. En el juego del ahorcado se utilizaron variables para almacenar la categoría seleccionada, el nivel de dificultad, la palabra oculta, las letras ingresadas por el jugador, los intentos restantes y el progreso de la palabra descubierta.

Entre las principales variables implementadas se encuentran:

- **cat:** almacena la categoría seleccionada por el usuario.
- **dif:** almacena la dificultad elegida (Fácil, Media o Difícil).
- **palabra:** guarda la palabra que el jugador debe adivinar.
- **mostrada:** lista donde se visualizan las letras descubiertas y los espacios ocultos.
- **usadas:** conjunto que almacena las letras ya utilizadas.
- **intentos:** contador que registra el número de errores del jugador.

Posteriormente se implementaron las estructuras condicionales mediante las instrucciones **if**, **elif** y **else**.

Estas estructuras permiten que el programa tome decisiones dependiendo de la información ingresada por el usuario.

Por ejemplo, cuando el jugador selecciona una dificultad, el programa verifica cuál fue la opción elegida:

- Si selecciona Fácil, se utilizan palabras cortas.
- Si selecciona Media, se utilizan palabras de longitud intermedia.
- Si selecciona Difícil, se utilizan palabras largas.

Las condicionales también permiten validar si una letra pertenece a la palabra, controlar los intentos restantes, verificar si el usuario ya ingresó una letra anteriormente y determinar cuándo el jugador gana o pierde la partida.

Gracias a estas estructuras el programa puede responder correctamente a todas las acciones del usuario.

Semana 5 – Implementación de Bucles

Durante la quinta semana se desarrollaron los ciclos repetitivos que permiten mantener el funcionamiento continuo del juego.

Se implementó principalmente el ciclo **while**, encargado de ejecutar el programa mientras el usuario continúe jugando.

El ciclo principal del juego permanece activo hasta que ocurre una de las siguientes condiciones:

- El jugador descubre toda la palabra.
- El jugador agota los seis intentos disponibles.
- El jugador decide rendirse.

Dentro de este ciclo también se actualiza continuamente el tablero, mostrando las letras descubiertas, las letras utilizadas y los intentos restantes.

Además del ciclo **while**, se utilizaron ciclos **for** para recorrer cada una de las letras de la palabra seleccionada.

Estos ciclos permiten:

- Mostrar la palabra oculta.
- Actualizar las letras acertadas.
- Buscar coincidencias entre la letra ingresada y la palabra.
- Mostrar el tablero actualizado.

El uso de los bucles permitió automatizar gran parte del funcionamiento del software, evitando repetir instrucciones manualmente.

Semana 6 – Diagrama de Flujo y Validación del Programa

Durante la sexta semana se elaboró el diagrama de flujo del sistema utilizando PSeInt.

El diagrama representa gráficamente el funcionamiento del algoritmo desde que el usuario inicia el programa hasta que finaliza la partida.

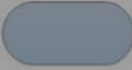
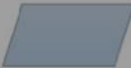
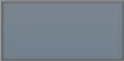
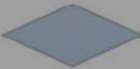
El proceso comienza mostrando el menú principal.

Posteriormente el usuario selecciona una categoría y un nivel de dificultad.

El sistema genera una palabra aleatoria de acuerdo con la dificultad seleccionada y crea el tablero inicial con espacios vacíos.

A continuación, inicia el ciclo principal del juego, donde el usuario puede:

- Ingresar una letra.
- Solicitar una pista.
- Rendirse.

Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Línea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso

Cada letra ingresada es validada para comprobar que sea un carácter alfabético y que no haya sido utilizada anteriormente.

Si la letra pertenece a la palabra, el tablero se actualiza mostrando todas sus posiciones.

Si la letra no pertenece a la palabra, el contador de intentos aumenta y el dibujo del ahorcado avanza un nivel.

El programa verifica continuamente si el jugador logró descubrir toda la palabra o si alcanzó el límite máximo de intentos.

Finalmente, el sistema muestra un mensaje indicando si el jugador ganó o perdió la partida y ofrece la posibilidad de jugar nuevamente o salir del programa.

Durante esta etapa también se realizaron pruebas de funcionamiento para verificar que todas las opciones del menú, las categorías, las dificultades y el sistema de pistas operaran correctamente sin presentar errores.

Semana 7 – Uso de GitHub y Control de Versiones

Durante la séptima semana se implementó el uso de GitHub como herramienta para el control de versiones del proyecto. Esta plataforma permitió almacenar el código fuente de forma segura, registrar cada uno de los cambios realizados y mantener un historial del desarrollo del software.

El uso de GitHub facilitó la organización del proyecto mediante la creación de un repositorio donde se almacenó el código del juego del ahorcado. Cada modificación realizada fue registrada mediante commits, permitiendo identificar fácilmente las mejoras incorporadas durante el desarrollo.

Además, GitHub permitió respaldar el proyecto, evitando la pérdida de información y facilitando futuras actualizaciones. Gracias al control de versiones fue posible mantener un desarrollo ordenado y documentado, siguiendo buenas prácticas de ingeniería de software.

Entre las ventajas obtenidas con GitHub se destacan:

- Respaldo permanente del código fuente.
- Registro histórico de cambios mediante commits.
- Organización del proyecto en un repositorio.
- Facilidad para realizar futuras mejoras.
- Posibilidad de trabajo colaborativo en versiones posteriores.

Semana 8 – Integración Final y Pruebas del Software

Durante la última semana se realizó la integración de todos los módulos desarrollados durante el semestre para obtener un software completamente funcional.

En esta etapa se ejecutaron diferentes pruebas para verificar el correcto funcionamiento del programa. Se comprobó que el sistema permitiera seleccionar correctamente la categoría, el nivel de dificultad y generar palabras aleatorias según las condiciones establecidas.

También se verificó que el programa validara adecuadamente las letras ingresadas por el usuario, evitando caracteres inválidos o letras repetidas. Se realizaron pruebas del sistema de pistas, del contador de intentos y de los mensajes de victoria y derrota.

Finalmente, se revisó la organización del código, la documentación del proyecto y la correcta ejecución del programa desde el menú principal hasta la finalización de cada partida.

Las pruebas demostraron que el software cumple con los objetivos planteados al inicio del proyecto y que todas las funcionalidades implementadas operan correctamente.

Explicación del Código Fuente

Importación de Librerías

El programa comienza importando las librerías random y os.

La librería random permite seleccionar palabras y categorías de forma aleatoria, haciendo que cada partida sea diferente.

La librería os se utiliza para limpiar la pantalla de la consola y ofrecer una mejor experiencia al usuario durante el desarrollo del juego.

Uso de Variables

Durante el programa se utilizan múltiples variables para almacenar información temporal.

Entre las principales se encuentran:

- palabra
- categoria
- dificultad
- intentos
- usadas
- mostrada
- letra

Cada una almacena información necesaria para controlar el estado de la partida.

Uso de Tuplas

Las tuplas se utilizaron para almacenar información constante, como los menús principales y los niveles de dificultad.

Al ser estructuras inmutables ofrecen mayor seguridad y organización.

Uso de Listas

Las listas se emplearon para almacenar las palabras disponibles dentro de cada categoría y para representar visualmente la palabra que el jugador debe descubrir.

Estas estructuras permiten modificar fácilmente su contenido conforme avanza la partida.

Uso de Diccionarios

Los diccionarios organizan las categorías del juego.

Cada categoría representa una clave y contiene una lista de palabras relacionadas.

Gracias a esta estructura resulta sencillo agregar nuevas categorías sin modificar el funcionamiento del programa.

Uso de Funciones

El programa fue dividido en funciones independientes para facilitar su mantenimiento.

Entre las funciones implementadas se encuentran:

- limpiar ()

- menú ()
- elegir categoría ()
- elegir dificultad ()
- obtener palabra ()
- ayuda ()
- tablero ()
- validar ()
- dibujo ()
- victoria ()
- fin ()
- jugar ()

Cada función realiza una tarea específica, evitando repetir código y mejorando la organización del programa.

Uso de Condicionales

Las estructuras **if**, **elif** y **else** permiten tomar decisiones durante toda la ejecución del juego.

Se utilizan para:

- Validar la opción seleccionada.
- Determinar la dificultad.
- Verificar si una letra pertenece a la palabra.
- Controlar la victoria o derrota.
- Comprobar si una letra ya fue utilizada.

Estas estructuras constituyen la base lógica del funcionamiento del programa.

Uso de Bucles

El programa utiliza dos tipos de ciclos.

El ciclo **while** mantiene el juego en ejecución hasta que el usuario gane, pierda o decida salir.

El ciclo **for** recorre las letras de la palabra para actualizar el tablero y comprobar las coincidencias entre la palabra y la letra ingresada.

Estos ciclos permiten automatizar el funcionamiento del software.

Validación de Datos

Antes de aceptar una letra ingresada por el usuario, el programa verifica que:

- Sea un único carácter.

- Sea una letra del alfabeto.
- No haya sido utilizada anteriormente.

Esta validación evita errores durante la ejecución del programa y mejora la experiencia del usuario.

ANÁLISIS DEL SOFTWARE

Descripción del Software

El software desarrollado consiste en un juego interactivo del ahorcado ejecutado desde la consola utilizando el lenguaje de programación Python. El objetivo principal del juego es que el usuario descubra una palabra oculta antes de agotar el número máximo de intentos permitidos.

El programa ofrece diferentes categorías de palabras, niveles de dificultad y un sistema de pistas que mejora la experiencia del usuario. Además, incorpora validación de datos, dibujos ASCII del ahorcado y un menú interactivo que permite reiniciar la partida o salir del programa.

El desarrollo del software se realizó aplicando programación estructurada, organizando el código en funciones independientes que facilitan su comprensión, mantenimiento y futuras mejoras.

ARQUITECTURA DEL SOFTWARE

La arquitectura del sistema fue diseñada de manera modular para separar cada una de las responsabilidades del programa.

Interfaz de Usuario

Corresponde al menú principal, donde el jugador interactúa con el sistema. Desde esta sección el usuario puede iniciar una nueva partida, seleccionar la categoría, elegir la dificultad e ingresar letras durante el juego.

Lógica del Negocio

En esta capa se encuentran todas las funciones encargadas de controlar el funcionamiento del juego.

Entre ellas se encuentran:

- Selección de categorías.
- Selección de dificultad.
- Generación aleatoria de palabras.
- Validación de letras.
- Sistema de pistas.
- Control de intentos.

- Verificación de victoria o derrota.

Datos

Los datos del sistema están almacenados mediante diccionarios, listas y tuplas.

Los diccionarios contienen las categorías del juego.

Las listas almacenan las palabras correspondientes a cada categoría.

Las tuplas contienen información constante como los menús y niveles de dificultad.

Servicios Transversales

Como servicios complementarios se utilizaron:

- Librería random para seleccionar palabras aleatorias.
- Librería os para limpiar la consola.
- GitHub para el control de versiones.

DIAGRAMA DE CASOS DE USO

El diagrama de casos de uso representa gráficamente la interacción entre el jugador y el sistema.

El único actor del sistema es el Jugador.

El jugador puede realizar las siguientes acciones:

- Iniciar una partida.
- Seleccionar una categoría.
- Elegir el nivel de dificultad.
- Ingresar letras.
- Solicitar una pista.
- Rendirse.
- Visualizar el resultado.
- Jugar nuevamente.
- Salir del programa.

Este diagrama permitió identificar claramente todas las funcionalidades que debía cumplir el software antes de comenzar la programación.

DIAGRAMA DE ARQUITECTURA

El diagrama de arquitectura muestra la organización interna del software.

En la parte superior se encuentra el usuario interactuando con la interfaz.

Posteriormente el sistema procesa las acciones mediante la lógica del negocio.

Finalmente se accede a las estructuras de datos que almacenan las categorías y palabras del juego.

La arquitectura modular permitió dividir el programa en funciones independientes, facilitando la organización del código y el mantenimiento futuro del software.

RESULTADOS OBTENIDOS

Al finalizar el desarrollo del proyecto se obtuvo un software completamente funcional que cumple con todos los objetivos planteados al inicio del semestre.

Durante las pruebas realizadas se comprobó que:

- El menú principal funciona correctamente.
- La selección de categorías opera sin errores.
- La dificultad modifica adecuadamente la longitud de las palabras.
- Las palabras se generan aleatoriamente.
- El sistema valida correctamente las letras ingresadas.
- No permite ingresar caracteres inválidos.
- No permite repetir letras ya utilizadas.
- El sistema de pistas funciona correctamente.
- El dibujo del ahorcado se actualiza conforme aumentan los errores.
- El programa detecta correctamente la victoria y la derrota.
- El usuario puede reiniciar la partida o salir del sistema.

Los resultados obtenidos demuestran que el software cumple satisfactoriamente con los requerimientos establecidos durante la etapa de análisis.

EJECUCION DEL PROGRAMA

```
=====
JUEGO DEL AHORCADO
=====

Bienvenido jugador
Adivina la palabra antes de perder

1. Jugar
2. Salir

Selecciona:
```

```
Selecciona: 1

ELIGE UNA CATEGORÍA
-----
1. PROGRAMACION
2. TECNOLOGIA
3. UNIVERSIDAD
4. ANIMALES
5. PAISES
6. DEPORTES
7. FRUTAS
8. COLORES
9. PROFESIONES
10. VEHICULOS
11. CATEGORIA ALEATORIA

Selecciona categoría: █
```

```
=====
INICIANDO PARTIDA
=====

Categoría seleccionada: COLORES
Dificultad: FACIL

El reto ha comenzado...
Palabra oculta lista

- - - -

Enter para comenzar...|
```

```
SELECCIONA DIFICULTAD

-----

1. FACIL
2. MEDIA
3. DIFICIL

Selecciona dificultad: 1|
```

```
Selecciona: 1

ELIGE UNA CATEGORÍA
-----

1. PROGRAMACION
2. TECNOLOGIA
3. UNIVERSIDAD
4. ANIMALES
5. PAISES
6. DEPORTES
7. FRUTAS
8. COLORES
9. PROFESIONES
10. VEHICULOS
11. CATEGORIA ALEATORIA

Selecciona categoría: 11|
```

```
-----

Categoría: COLORES
Dificultad: FACIL
Palabra: _ _ U L
Letras usadas: L U
Intentos restantes: 6

-----

1. Letra
2. Pista
3. Rendirse

Selecciona: |
```

```
-----

Categoría: COLORES
Dificultad: FACIL
Palabra: _ _ U L
Letras usadas: L U
Intentos restantes: 6

-----

1. Letra
2. Pista
3. Rendirse

Selecciona: 1

→ Letra: A|
```

```
-----

Categoría: COLORES
Dificultad: FACIL
Palabra: A _ U L
Letras usadas: A L U
Intentos restantes: 6

-----

1. Letra
2. Pista
3. Rendirse

Selecciona: |
```

REINVENTEMOS
EL FUTURO

```
¡VICTORIA!  
  
Palabra: AZUL  
  
1. Jugar otra vez  
2. Menú principal  
3. Salir  
  
Selecciona: █
```

```
HAS PERDIDO  
  
Palabra: PERA  
  
1. Jugar otra vez  
2. Menú principal  
3. Salir  
  
Selecciona: █
```

```
HAS PERDIDO  
  
Palabra: PERA  
  
1. Jugar otra vez  
2. Menú principal  
3. Salir  
  
Selecciona: 3  
  
¡Gracias por jugar! Esperamos verte pronto.  
PS C:\Users\ANABEL\Documents\PROYECTO FINAL> █
```

IMPLICACIONES Y LIMITACIONES

Aunque el software cumple con todos los objetivos planteados, presenta algunas limitaciones.

Entre ellas se encuentran:

- El programa funciona únicamente desde la consola.
- No almacena estadísticas de los jugadores.
- No posee autenticación de usuarios.
- Las palabras son almacenadas manualmente dentro del código.
- No utiliza una base de datos.

Estas limitaciones pueden solucionarse en futuras versiones del proyecto.

MEJORAS FUTURAS

Como trabajo futuro se propone:

- Desarrollar una interfaz gráfica utilizando Tkinter o PyQt.
- Incorporar una base de datos para almacenar puntuaciones.
- Implementar un sistema de usuarios.

REINVENTEMOS
EL FUTURO

- Agregar nuevos niveles de dificultad.
- Incrementar el número de categorías.
- Permitir partidas multijugador.
- Incorporar sonidos y animaciones.
- Mostrar estadísticas de partidas ganadas y perdidas.
- Generar un ranking de jugadores.

CONCLUSIONES

1. El desarrollo del proyecto permitió aplicar los conocimientos adquiridos durante toda la asignatura de Lógica de Programación.
2. La utilización de variables, estructuras condicionales, ciclos repetitivos, funciones y estructuras de datos permitió construir un software organizado y funcional.
3. La elaboración del pseudocódigo y de los diagramas facilitó la planificación del algoritmo antes de comenzar la programación.
4. GitHub permitió administrar correctamente el desarrollo del proyecto mediante el control de versiones y el respaldo del código fuente.
5. El proyecto fortaleció las habilidades de análisis, diseño, programación y resolución de problemas, evidenciando la importancia de seguir una metodología organizada durante el desarrollo de software.

RECOMENDACIONES

- Continuar mejorando el programa incorporando nuevas funcionalidades.
- Implementar una interfaz gráfica para mejorar la experiencia del usuario.
- Mantener el uso de GitHub para controlar futuras versiones del software.
- Incrementar el número de palabras y categorías disponibles.
- Incorporar una base de datos para almacenar estadísticas y puntuaciones.
- Documentar cada nueva funcionalidad desarrollada.
- Aplicar pruebas periódicas para garantizar el correcto funcionamiento del sistema.

BIBLIOGRAFÍA

Python Software Foundation. (2024). Python Documentation. <https://docs.python.org/3/>

GitHub. (2024). GitHub Docs. <https://docs.github.com/>

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (9.^a ed.). McGraw-Hill.

Sommerville, I. (2017). Ingeniería de software (10.^a ed.). Pearson.

Joyanes Aguilar, L. (2018). Fundamentos de Programación. McGraw-Hill.

REINVENTEMOS
EL FUTURO